

TLM 3.x Interview Master Gap & Deep-Dive Guide

This is a gap/deep-dive add-on to the existing TLM interview PDF. It avoids repeating generic definitions unless the detail is needed for follow-up-proof interview answers. It is written for a 5+ year Singapore banking SmartStream TLM 3.x Techno-Functional Consultant interview.

Accuracy rule used throughout: Physical object name is implementation/version/client dependent - verify against the client schema/configuration. Do not invent physical SmartStream TLM table names, MI syntax, ServiceMix filenames, proprietary commands, APIs, queue table names or client-specific configuration screens. Use logical objects, prove the layer, and then map to verified client schema/configuration.

Baseline assumed: The earlier TLM interview guide already covers broad TLM architecture, static setup, matching basics, MI/DMW, SQL, Linux and production support. This master gap guide concentrates on interview-breaking areas: logical database architecture, exact data-flow reasoning, static-vs-transaction DB impact, message feed/header controls, cash reconciliation depth, mirror/double-entry, proposal triggering, queue/MMQ reasoning and evidence-driven troubleshooting.

Part 1 - Database Table / Logical Data Architecture

Before answering any 'which table?' question, say: Physical object name is implementation/version/client dependent - verify against the client schema/configuration. Then answer at the logical area level and explain how you would discover the real object from SmartSchema metadata, data dictionary, verified support SQL, vendor documentation, deployment runbook or existing application logs.

Logical area | Purpose | Stores | When populated | Trigger | Parent/child | Identifiers | Before | After | How to query | Diagnose | Failure | Interviewer asks | What NOT to claim

Static/configuration data | Reference and model configuration | sets, recon definitions, sources, feeds, rules, workflows, users/roles | When configuration is created/changed/approved | Admin/Smart Studio/Recon Admin/TLM Control change | Parent of transaction processing | codes, effective dates, active flag, owner, maker/checker | Requirement/design | Feed/load/item association | Query active config by code/effective date | Wrong or inactive setup causes load association, matching or visibility issues | No matching, wrong queue, data not visible | Which config object changed? | Do not claim transaction tables update just because static was created

Set | Operational/configuration container | set code/name, ownership, active status, related recons/accounts | During static setup | Set creation/approval | Parent to recon/account grouping | set_code, set_id, business unit | Requirement | Reconciliation setup | Find active set and linked recons/accounts | Check whether item account belongs to set | Records route to wrong set or not seen | What is Set vs Reconciliation? | Do not call Set a source file

Reconciliation | Concrete reconciliation process | recon code, class, data sources, rules, workflow, triggers | Static build/change | Recon creation/approval | Child of set/class; parent of item association/runs | recon_id, recon_code, class, status | Set/class | Load association/workflow | Query by recon code and effective status | Wrong recon means correct data loaded but not matched as expected | Items loaded under wrong recon | Mandatory recon fields? | Do not say recon is only a dashboard

Reconciliation Class | Model/type of recon behavior | cash/position/securities/inter-system semantics and allowed dimensions | Model setup | Class/model selection | Parent template for reconciliations | class_code, class_type | Solution design | Reconciliation | Verify recon class and allowed dimensions | Wrong class leads to wrong proof/matching dimensions | Cash fields absent in position model or vice versa | Class vs Data Source? | Do not equate class with source system

Data Source | Logical side/input role | Ledger, Statement, Custodian, Broker, Internal Holding | Configured before feed/item association | Recon/data-source setup | Child of recon; parent target for feeds/items | data_source_code, side, source role | Reconciliation | Message feed mapping | Count items by data source and recon | Both sides mapped as same source or missing side | Only one side has items | Why both class and source? | Do not call it the physical database source

Message Feed | Inbound stream definition | feed code, source, message type, format, schedule, mapping, duplicate controls | Feed configuration and at each load as feed metadata | Integration/loader pickup | Child of source/data source; parent of load batch | feed_id, feed_code, short_code, format, active flag | Source/message type | File header/load batch | Query loads/items by feed | File loads to wrong side/recon or rejects | Wrong mapping or duplicate status | What fields make a feed? | Do not invent feed table name or parser filename

Account | Business control account | nostro/depot/GL/account aliases, currency, BU, effective dates | Static setup and account mapping | Account creation/approval or lookup sync | Parent for items/balances/exceptions | account_id, account_no, alias, currency, BU | Source/account requirement | Item association/proof | Query account alias active for source account | Wrong account causes invisibility/wrong breaks | Items loaded under suspense or wrong account | Mandatory account static? | Do not update account tables directly without approved process

Source/reference data | Master/reference data used for enrichment | source systems, security aliases, currency, customer, BU, calendar | Static load/sync | Reference feed/admin update | Parent lookup for MI/DMW | source_code, asset_id, currency, effective dates | Upstream reference | Mapping/enrichment | Lookup by source value and effective date | Lookup miss/reject/wrong derivation | Rejects or false breaks | What if lookup fails? | Do not assume all reference data is manually maintained

Message Type | Format/business message category | CSV ledger, MT940, MT950, camt, MT535, proprietary statement | Feed setup | Format definition selected | Parent to feed/parser validation | message_type_code, version, format | Source file design | Parser/mapping | Query feed by message type | Wrong parser/validation | Malformed file despite arrival | What is message type? | Do not invent SWIFT parser internals

File Header / File Metadata | File-level control information | file name, path, size, checksum, business date, record count/trailer count, arrival time | At file pickup/registration | File arrival/polling | Parent to batch/load detail | file_id, file_name, feed_id, business_date, hash | Landing directory | Load batch | Query file metadata by file name/date | Wrong date/count/hash/duplicate | File present but not processed or duplicate | How prove file arrived? | Do not assume metadata exists in every implementation; some use logs/files

Message Header | Message-level envelope/header | message reference, sender, receiver, account, statement sequence, date | During parsing | Parser reads message | Child of file; parent to detail records | message_id, message_ref, sequence, sender | File header | Load detail/item | Query by message reference or statement sequence | Wrong account/business date/sequence | Records under wrong statement/account | Header vs file header? | Do not invent MT field-to-table mapping

Load/Batch Control | Execution/control record for a loader run | load id, feed, file, start/end time, status, accepted/rejected counts | At loader start/update/end | MI/DMW/ETL execution | Parent to load detail/items/rejects | load_id, batch_id, status, counts | File/message metadata | Load detail/BDR item | Query latest load by file/feed/status | Failed/partial/duplicate load | Status failed, accepted < expected | Which table first? | Do not say BDR item is always first write

Load Detail | Record-level load trace | record number, source values, target keys, status, line number | During record processing | Parser/mapping per record | Child of load batch; parent/reference to item/reject | load_id, line_no, source_record_id | Load control | Item/reject | Query by load id + line/reference | Specific line failed or mapped wrong | Only some records missing | How trace one transaction? | Do not assume every client stores raw detail; logs/reject files may hold it

Reject/Error/Failure information | Rejected records and load errors | error code/text, field, raw value, line, validation rule, stack/DB error | When parser/mapping/DB validation fails | Validation or DB failure | Child of load/batch/detail | load_id, line_no, error_code, field_name | Load detail | Correction/reload | Query rejects by load/file/error | Partial/full rejection | Accepted+rejected mismatches | How identify rejects? | Do not hide rejects by defaulting mandatory fields

BDR/business item | Canonical reconcilable item | item id, source lineage, account, amount/qty, currency, dates, ref, status | After successful validation/mapping | Record accepted/committed | Child of load/feed/account; parent of matching/workflow/exception | item_id, source_txn_id, load_id, account, ref |

TLM 3.x Interview Master Gap & Deep-Dive Guide

Load detail | Recon association/workflow | Query by source ref/file/account/date | No item/wrong item values | Loaded but invisible/unmatched | Which item did feed generate? | Do not invent BDR table names
Reconciliation association/status | Links item/account/day to recon lifecycle | recon_id, item_id, side/source, status, business date | After item creation/static association | Association rule/account/feed mapping | Child of item/recon; parent of matching | item_id, recon_id, data_source, recon_status | BDR item/static | Workflow/scope | Query item by recon/status/source | Item loaded but not in recon | No matching because ineligible | How can load succeed but recon fail? | Do not assume load success means recon eligibility
Matching/pass results | Diagnostics from match pass evaluation | pass name/order, candidate group, criteria result, quality result | During matching run | Matching service evaluates population | Child of matching run/items; parent of proposal/match | run_id, pass_id, item_id/group_id, result | Scope/population | Proposal/match/exception | Query by run/item/pass result | No pass or failed pass | No proposal despite candidates | Why pass failed? | Do not say unmatched means no data without checking pass diagnostics
Proposal | Candidate match group before final/auto/manual match | proposal_id, items, quality, status, created by pass | When pass quality produces proposal | Matching result creation | Child of pass results; parent of match or manual decision | proposal_id, item_ids, status, quality | Pass quality | Match/manual review | Query proposals by item/recon/status | Pending/not visible/not approved | Proposal exists but no match | Proposal vs match? | Do not say proposal always equals final match
Match result/history | Final accepted match state/history | match_id/group, item links, matched status, user/system, timestamp, audit | Auto match/manual approval | Proposal acceptance or automatic match | Child of proposal/items; affects item status/exceptions | match_id, item_id, matched_ts, match_type | Proposal | Exception closure/audit/reporting | Query item-to-match lineage | False match/duplicate match/missing match | Items wrongly closed or still open | How prove match happened? | Do not update match status manually outside controls
Workflow | Process lifecycle definition and runtime state | workflow_id, state, transitions, work item, assignment context | Static setup and runtime initiation | TLM Control/initiation/actions | Parent/child to items/exceptions/queues | workflow_id, object_id, state, transition | Recon association | Queue/exception/view | Query workflow state by item/exception | Stuck state/no transition | Loaded item not visible or not processed | What workflow state? | Do not claim exact workflow table name
Initiation | Event/rule starting workflow/matching lifecycle | initiated object, rule/trigger, timestamp, result | After item/recon eligibility or schedule/manual trigger | Load completion/schedule/manual action | Creates workflow/queue entries | initiation_id, object_id, trigger_type | Recon association | Queue/MMQ | Query initiation status for item/recon/date | No queue/no matching | Items loaded but idle | What triggers matching? | Do not confuse initiation with data load
MMQ/workflow queue | Logical queued work for processors | object_id, queue type, status, priority, retry count, next run time | When workflow/matching/integration work is queued | Initiation/trigger/action | Child of workflow/initiation; consumed by processor | queue_id, object_id, status, retry, created_ts | Initiation | Processor/matching/workflow transition | Query queue status/age/backlog | Stuck/backlog/poison record | Old queue age, retries increasing | Who polls queue? | Do not invent MMQ expansion/table name
Queue history/log | History of queue attempts/transitions | attempt count, error, consumer, start/end, status changes | Each queue processing attempt | Processor picks work | Child of queue entry | queue_id, attempt_no, error_text | Queue entry | Success/failure/retry | Query by queue id and attempts | Repeated failure/poison record | Same error each retry | How prove retry? | Do not restart repeatedly without root cause
Exception/break | Actionable unresolved reconciliation issue | exception_id, type, reason, resolution, owner, inbox, SLA, comments | When matching leaves unresolved issue or user raises one | Match/break rules/workflow/manual | Child of item/match result; parent to workflow/audit | exception_id, item_id/match_id, status, owner | No match/failed pass | Owner/investigation/resolution | Query open exception by item/ref/account | Not raised/wrong queue/wrong owner | Business does not see break | Exception lifecycle? | Do not call every unmatched item an exception unless configured
Balance/proof | Balance and proof control | opening/closing balance, movements, internal/external balance, proof status | During balance load/proof calculation | Balance feed/proof job/matching event | Parent/peer to items and exceptions | account, currency/asset, business_date, balance_type | Load/item movements | Proof exception/dashboard | Query by account/date/currency/asset | Proof mismatch despite matched txns | Unbalanced account | What is balance table? | Do not name physical balance table without verification
Audit/history | Control evidence | user/system action, timestamp, before/after, comments, approvals | Every controlled operation | UI/action/service/config change | Child/history of all major objects | audit_id, object_id, action, user, ts | Any lifecycle event | Reports/control evidence | Query audit by object/ref/user/time | Missing evidence or unauthorized change | Cannot explain who changed what | How prove it? | Do not bypass audit with direct DB updates
Security/user/role | Access control | users, roles, groups, inbox access, dashboard permissions | User provisioning/static setup | IAM/admin change | Controls visibility/actions | user_id, role_id, group, permission | Org setup | TLM View/workflow | Query user's roles/groups/inboxes | User cannot see data or cannot act | DB has data, UI missing for one user | Why user can't see? | Do not assume data missing before checking role/filter
TLM View/dashboard metadata | Dashboard/view configuration | widgets, columns, filters, sorts, groupings, shared views, role visibility | Dashboard setup/change | TLM View/admin action | Reads underlying recon data/security | dashboard_id, widget_id, column, filter, role | Data model/security | User display/reporting | Compare dashboard filter with DB query | UI count differs from DB | False missing records | How add column/header? | Do not confuse dashboard column label with DB column name

Logical dependency diagram

Static/reference foundation

Set

- > Reconciliation Class
- > Reconciliation
 - > Data Source(s)
 - > Message Feed(s)
 - > Account/static eligibility
 - > Workflow + Initiation + Proposal Trigger
 - > Scope + Population + Match Pass + Pass Quality

Runtime data lineage

Source System

- > File/Message

TLM 3.x Interview Master Gap & Deep-Dive Guide

- > File Header/Metadata
 - > Message Header
 - > Load/Batch Control
 - > Load Detail
 - > Reject/Error OR BDR Business Item
 - > Reconciliation Association/Status
 - > Workflow Initiation
 - > MMQ/Workflow Queue
 - > Scope
 - > Population
 - > Match Rule/Pass Quality
 - > Proposal
 - > Match Result/History OR Exception/Break
 - > Audit/History
 - > TLM View/Dashboard

Balance/proof lineage

Source balance/statement

- > Load/Batch
 - > Balance/Proof logical area
 - > Proof status
 - > Exception/Break if unbalanced
 - > Workflow/Audit/TLM View

Part 2 - Complete DB Data Flow and Reverse Troubleshooting

Step | What comes in | What changes/logical storage | Next trigger | Can fail | Prove success | Prove failure | SQL investigation | Linux investigation | App log investigation

SOURCE SYSTEM | Extract/message/event from ledger/bank/custodian | No TLM change yet unless direct DB extraction | Source scheduler/API/file event | Extract not produced, wrong date | Source control totals and upstream confirmation | No file/no control total | N/A or upstream staging query | Check source file path only if accessible | Source/scheduler logs

FILE/MESSAGE | Delimited/fixed/XML/SWIFT/proprietary payload | Raw file/message available | Transfer job or middleware route | Malformed, zero byte, wrong filename | File exists, size > 0, expected count/date | Header/trailer mismatch, bad parser | File metadata/load metadata once picked | ls -l, wc -l, head, tail, grep ref | Transfer/parser logs

TECTIA/SFTP | Secure file transfer | Transfer audit/log; landing file | SFTP completion/marker/rename | Permission/key/network/partial transfer | Complete file timestamp, owner, checksum | Partial/late/duplicate | Later file metadata by filename | ls -ltr, stat, checksum | Tectia/SFTP logs

LANDING DIRECTORY | File ready for pickup | Filesystem state; maybe marker file |

Polling/scheduler/ServiceMix | Wrong path/permission/disk | Loader user can read file | File ignored | No load batch row | find latest, df -h, ls -l | Integration logs

SERVICEMIX/MIDDLEWARE | Route file/message to loader | Route/queue/integration log | Polling/event/route | Route inactive, endpoint down | File moves to processing/archive and loader called | File stuck in inbound | Integration queue/load metadata | ps/log/tail where runbook says | ServiceMix/container logs

DMW/MI/ETL | Parse, map, transform, validate, lookup | Mapping run, rejects, load metadata | Loader execution | Mapping/lookup/datatype errors | Accepted + rejected = source count | Rejects/full batch failure | Load/reject logical areas | grep ref in reject/log | MI/DMW/ETL logs

FILE/MESSAGE HEADER | Capture file/message envelope | file id, msg id, date, count, sequence, sender/account | Parser registration | Wrong date/account/count | Header matches expected business date/account | Successful load to wrong account/date | Header/logical metadata query | head/tail/source file | Parser logs

LOAD/BATCH | Control loader execution | load id, status, counts, timestamps | Loader starts/commits | Failed/partial/duplicate | Status success; counts correct | Status failed or incomplete | SELECT by file/feed/status | N/A | Loader DB/app logs

BDR/ITEM | Create canonical item | item id, account, ref, amount/qty, currency/date, lineage | Accepted record | No item/wrong value | Query by source ref/load/account | Missing/wrong item | SELECT item by ref/file/load | grep source ref | Mapping/item creation logs

STATIC ASSOCIATION | Attach item to recon/account/source/class | recon id, data source, account status | Account/feed/recon rules | Wrong/inactive static | Item has expected recon/source/account | Loaded but not eligible | SELECT by item and recon/status | N/A | Association/workflow logs

RECON | Item is in recon lifecycle | recon status/business date | Association success | Wrong status/business date | Eligible/open for matching | Not picked in scope | Query recon status/source | N/A | Recon logs

WORKFLOW INITIATION | Start process for item/recon/day | initiation status/workflow state | Load completion/schedule/manual trigger | Initiation rule false/service down | Initiated timestamp/state exists | No queue/no match | Query initiation/workflow state | N/A | TLM Control/workflow logs

QUEUE/MMQ | Queue work for processor | queue status, attempts, retry, created time | Initiation/proposal trigger | Backlog/stuck/poison/locks | Queue processed or age acceptable | Old queue/retry errors | Queue status/age query | N/A | Queue/processor logs

SCOPE | Select eligible items | scope run/candidate count | Matching service run | Too narrow/too broad | Expected candidate count | No candidates | Query scoped item statuses/recon/date | N/A | Matching diagnostics

POPULATION | Group candidates | candidate groups | Scope output | Bad grouping keys/combinatorial explosion | Expected counterpart grouped | Candidates separated/wrong group | Pass diagnostics if stored/logged | N/A | Matching logs

MATCH RULE/PASS QUALITY | Compare and classify | pass results/quality | Population evaluation | Wrong tolerance/ref/date/sign | Pass result explains match/no-match | Unexpected fail/false match | Query pass/result by item/run | N/A | Matching logs

PROPOSAL | Suggested/perfect/pending candidate group | proposal id/status/items | Pass quality success/pending | Not generated/not visible/not approved | Proposal exists if expected | No proposal despite candidates | Query proposal by item | N/A | Matching/UI logs

MATCH | Final accepted match | match id/history/item status | Auto match/manual approval | False/duplicate/missing match | Item status matched and audit exists | Item open or wrongly matched | Query match history by item | N/A | Audit/matching logs

EXCEPTION | Actionable break | exception status/type/inbox/owner | No match/failed proof/manual raise | Not raised/wrong owner | Open exception visible to correct team | Break invisible | Query exception by item/ref/account | N/A | Exception/workflow logs

AUDIT | Evidence of actions | who/what/when | Every controlled event | Audit missing/unauthorized action | Audit trail complete | Cannot prove changes | Query audit by object/user/time | N/A | Audit logs

TLM VIEW | Operational display | dashboard metadata, filters, user views | User opens view | Filter/security/cache mismatch | UI count matches DB under same criteria | User says missing | Compare DB count with dashboard filters | N/A | Web/UI logs

Reverse troubleshooting path: TLM View missing -> Source

1. TLM View: capture user's dashboard, filters, business date, role, account, status. Compare with admin/user-independent view.
2. DB item count: use same criteria logically against item/recon/status areas. If DB has rows but UI does not, check dashboard/security/cache/view metadata.
3. Exception: if user expects a break, query logical exception area by reference/account/date. If absent, check match/proposal/status.
4. Match: query match history by item/reference. If matched, explain no open break. If false match, check rule/pass/audit.
5. Proposal: proposal may be pending/suggested/not approved. Query proposal logical area.
6. Queue: if no proposal/match, check queue entry, age, retry, processor errors.
7. Workflow: confirm initiation and current state. Loaded data may be idle if initiation failed.

8. Item: query BDR/business item by source ref, file, account and date. If no item, go to load/reject.
9. Load: check batch status, accepted/rejected counts, partial load and mapping errors.
10. Feed: confirm file picked by correct feed and mapped to expected data source/recon.
11. File: confirm file arrived, size, record count, checksum, naming convention, timestamp.
12. Source: if file/record absent, work with upstream using control totals and source job evidence.

Part 3 - Static Creation from Zero

Object | Purpose | Mandatory logical fields | Dependencies | Expected result | Common error | DB logical impact | How to test

Set | Container for recon package | unique code/name, active/effective date, owner/BU | BU/security, naming standard | Approved and visible | Duplicate/inactive/wrong BU | Static metadata only | Create dummy recon/account link; verify visibility

Reconciliation | Specific process | recon code/name, set, class, sources/sides, business date/calendar, workflow, triggers | Set/class/accounts/data sources | Recon active and eligible | Wrong class or inactive | Recon config metadata | Load sample item and check association

Reconciliation Class | Recon behavior model | class/type, dimensions, proof model if applicable | Solution design | Fields/rules align to cash/position | Cash built on position model | Model metadata | Confirm available fields/rules

Data Sources | Recon sides | side/code/name, recon link, source role | Recon/class | Ledger and Statement separate | Both feeds mapped to same side | Data-source metadata | Load one record per side and count by source

Message Feeds | Inbound stream | feed code, short code, source, message type, file pattern, schedule, mapping, duplicate/reject controls, target data source | Data source, source, message type | File picked and linked to correct side | Wrong feed/wrong mapping | Feed/static + load metadata at runtime | Unit test file pickup and counts

Accounts | Business control account | account id, source account alias, currency, BU, customer, active dates, recon/set association | BU/source/currency/reference data | Items map to expected account | Rejects or wrong account breaks | Account static/reference | Lookup test with sample source account

Workflow | Lifecycle/routing | workflow name, states, queues/inboxes, roles, SLAs, exception types | Recon/security | Items/exceptions route correctly | Stuck/invisible breaks | Workflow config metadata | Trigger sample break and observe queue

Initiation | Start workflow/match | trigger event, conditions, schedule/manual option, target queue | Workflow/recon/feed availability | Queue entry created as expected | Loaded data idle | Initiation/workflow metadata | Load sample and verify initiated

Scope | Eligibility | recon/status/date/account/source criteria | Item/status fields | Expected candidate count | No candidates/too many | Match config metadata | Run matching with known data

Population | Candidate grouping | grouping keys, side requirements | Scope/match fields | Counterparties grouped | Candidates separated/explosion | Match config metadata | Inspect diagnostics/proposal

Match Rule | Comparison logic | amount/date/ref/currency/account/sign/tolerance | Population fields | Expected pass/fail | False match/no match | Match config metadata | Positive/negative test cases

Pass Quality | Categorize quality | perfect/suggested/pending thresholds, mandatory checks | Match rule results | Correct auto/suggested behavior | Weak auto-match | Quality metadata/results at runtime | Test exact, tolerant, mismatch

Proposal Trigger | When to propose | any/all/always/never/scheduled/feed availability/manual | Feeds/workflow/service | Proposals generated at right time | Premature/missing proposals | Trigger config and runtime queue | Daily/monthly feed simulation

Static vs transaction data: Static creation changes configuration/reference/model metadata. It should not by itself create business items, match records, proposal records or exceptions. Runtime transaction data is created only when files/messages/extracts load, items are created, workflow initiates and matching/exception processing runs. If an interviewer asks what happens in DB when static is created: answer that configuration/static/reference logical areas change, audit/maker-checker history may change, and downstream transaction tables are not expected to receive records until runtime processing occurs.

Part 4 - Message Feed / Message Header Deep Dive

Concept | Deep interview answer | Failure pattern | Evidence
 Feed code | Unique identifier for the inbound stream; connects file pattern/parser/mapping to data source/recon. | Wrong feed selected or no pickup | File metadata and load batch feed id/code
 Short code | Client-friendly abbreviation sometimes used in filenames/logs/UI; exact use is implementation dependent. | Support team searches wrong code | Runbook/feed setup
 Message Type | Format/business message: CSV ledger, MT940, MT950, camt, MT535 etc. | Parser mismatch | Feed setup/parser logs
 File identification | Filename pattern, source, business date, sequence, extension, marker/checksum rules. | Wrong filename not picked | ls/find plus load metadata
 Header | File/message envelope: date, account, statement ref, sender/receiver, sequence. | Records load under wrong account/date | Header metadata/parser log
 Trailer | Record count/control total/end marker. | Partial load or count mismatch | Trailer vs load accepted/rejected
 Record count | Control total to compare with accepted+rejected or expected message count. | Partial/zero/overload | wc -l, trailer, batch counts
 Business date | Date used for recon eligibility and matching windows; may differ from file arrival date. | Data loaded but not in today's view | Header/load/item business date
 Source/account | Header or detail values mapped to TLM source/account; may require alias lookup. | Wrong account/source or rejects | Account alias/static + item
 Feed-to-data-source association | Determines side role such as Ledger or Statement. | Correct load but wrong recon side | Items count by feed/data source
 Proposal triggering | Feed availability may be part of trigger; load success does not guarantee proposal. | No proposals after load | Trigger/workflow/queue/matching logs
 Duplicate file control | Usually by filename, source batch, hash, sequence, business date or client-specific key. | Duplicate items/matches | Duplicate load metadata/business keys
 Partial file | File transferred incomplete or parser processed partial records. | Accepted+rejected less than expected | Checksum/trailer/file size
 Malformed file | Bad delimiter, bad date/amount, missing header/trailer, invalid SWIFT structure. | Full/partial rejection | Parser errors/rejects
 Zero-byte file | No business records; should generally fail/alert rather than produce false success unless agreed. | Zero loaded records | ls -l size, load count 0
 Wrong feed | Filename/route maps file to another feed. | Items under wrong data source/recon | File metadata feed code
 Wrong mapping | Field-level target values incorrect. | False breaks or rejects | Compare raw vs item fields

A file can load successfully but still fail reconciliation association when feed-to-data-source mapping is wrong, account alias is missing/inactive, business date falls outside recon scope, recon/account is inactive, reconciliation class does not support required fields, or initiation/trigger conditions are not satisfied. Prove it by showing successful load status and item count, then showing missing/wrong recon association/status or workflow initiation.

Part 5 - Cash Reconciliation Master

Cash concept | Interview-ready deep explanation

Nostro | Our account with another bank; in recon, internal nostro ledger is compared to external bank statement.

Vostro | Their account with us; may appear in correspondent banking, not the same control perspective as nostro.

Ledger/Internal GL | Bank's internal postings; may include debit/credit accounting legs and sign conventions.

Statement/Bank statement | External bank view of account activity and balances.

MT940 | Customer statement style cash statement commonly used for end-of-day/intraday transaction/balance reporting; parser details are client-specific.

MT950 | Statement message/account reporting; usage differs by bank/counterparty.

CAMT equivalents | ISO 20022 camt.052/camt.053/camt.054 style cash reports/statements; mapping is implementation dependent.

Transaction | Movement/posting line to match.

Balance | Opening/closing/available/interim control amount.

Opening balance | Starting balance for proof.

Movement | Debit/credit transaction amount affecting balance.

Closing balance | Ending balance after movements.

Proof | Opening + net movements = closing; compare internal vs external.

Value date | Date funds are valued; key for cash matching.

Posting date | Date record posted/accounted.

Transaction date | Date event occurred; may differ from posting/value date.

Amount/currency | Core cash dimensions; sign normalization is critical.

Reference/narrative | Identifiers/narrative used for matching; often requires normalization or extraction.

Debit/credit | Direction from each side's perspective; must be normalized before comparing.

Bank charges | Fee difference may require tolerance/write-off/exception depending policy.

FX | Cross-currency or converted amount matching; requires agreed rate/source.

Reversal | Opposite posting correcting earlier transaction; avoid duplicate false matches.

Timing difference | One side appears earlier/later; often aged unmatched or pending.

Duplicate | Same file/record/posting loaded twice or source duplicate.

Missing item | One side absent due to late feed, reject, upstream issue or wrong mapping.

Tolerance | Controlled permitted difference; should be governed and audited.

Manual match | User-approved match under permissions and audit.

Write-off | Controlled accounting/business resolution for small differences.

Exception | Break investigation object with owner, SLA, reason, resolution and audit.

Cash recon proof example: external opening 100,000 + credits 25,000 - debits 15,000 = external closing 110,000. Internal ledger movements may all match transaction-to-transaction, but if internal opening is 99,500 or a bank charge is treated differently, proof can still break. Therefore a senior answer always checks both transaction matching and balance proof.

Part 6 - Mirror vs Double Entry - Extreme Depth

Terminology warning: Mirror and Double Entry are used differently across banks. Physical object name is implementation/version/client dependent - verify against the client schema/configuration. Explain the accounting/reconciliation concept first, then ask how the client implements it in TLM configuration/mapping/matching.

Scenario | Raw records | Normalization/mirror reasoning | Matching implication | Risk

1 Statement CREDIT 10,000 vs Ledger DEBIT 10,000 | External statement sees money into nostro as credit; internal ledger may represent our asset account as debit increase. | Normalize both to same economic direction/amount, e.g., +10,000 incoming cash. Mirror concept is opposite sign/perspective, not duplicate source data. | After sign normalization, one statement item can match one ledger cash leg if account/ref/date/currency pass. | Without sign normalization it appears amount sign mismatch.

2 Statement DEBIT 10,000 vs Ledger CREDIT 10,000 | External statement sees outgoing as debit; internal cash asset decrease may be credit. | Normalize to -10,000 outgoing cash or compare absolute amount with direction rule. | Match same economic event, not raw sign equality. | Wrong sign rule creates false breaks or false matches.

3 Ledger has DR + CR accounting legs | A payment may debit expense/receivable and credit cash, or debit cash and credit clearing depending event. | Double entry is accounting completeness. Reconciliation normally selects the relevant cash/nostro leg or derives a comparison item; it should not blindly match both GL legs to one statement line. | Statement transaction should match the cash leg or controlled aggregated representation, not two unrelated accounting legs. | One mirror item incorrectly matching two raw GL legs inflates match count and hides imbalance.

4 Multiple ledger postings correspond to one statement transaction | Bank statement single 10,000; ledger has four postings of 2,500 or debit/fee split. | Use one-to-many/netting/population rules with account/ref/date/currency and pass quality; preserve source lineage. | Proposal may be one statement item to multiple ledger items if aggregate amount and quality pass. | Weak grouping may match unrelated postings. Partial double-entry posting | Only one accounting leg arrives before the other or cash leg missing. | Treat as source/timing/accounting issue; do not force mirror match. | May create pending/unmatched/exception until missing leg arrives. | False match hides unbalanced ledger.

Unbalanced posting | DR and CR totals do not balance in source ledger. | This is accounting control issue before/alongside TLM matching. | Do not reconcile by matching one side to statement while ignoring source imbalance unless agreed. | Operational risk and audit issue.

Wrong reference | Ledger cash leg has wrong ref; statement correct. | Reference normalization/masking may help only if policy allows; pass quality should lower confidence. | Suggested/manual not perfect if reference missing/wrong. | Auto-matching weak reference can hide upstream issue.

Duplicate posting | Same GL cash leg appears twice. | Duplicate detection by business key/source batch before matching. | One statement should not match duplicate ledger unless aggregate rule intentionally expects two items. | False many-to-one hides duplicate.

Timing difference | Ledger today, statement tomorrow/month-end. | Keep unmatched/pending based on ageing and trigger strategy. | Do not force match before opposite side exists. | False exceptions if proposal trigger not aligned.

Key spoken answer: Mirror is about reconciling opposite perspectives or derived expected comparison; double entry is the accounting rule that every event has debit and credit legs. In TLM, the implementation may be sign normalization, derived/virtual comparison items, selected ledger leg extraction, aggregation or client-specific mirror configuration. I verify the actual implementation before claiming behavior.

Part 7 - Matching Deep Dive

Concept | Deep explanation

Exact | All configured fields equal after normalization.

One-to-one | One item each side.

One-to-many | One item matches aggregate of many on other side.

Many-to-one | Many source items match one target item.

Many-to-many | Aggregated groups on both sides match.

Amount | Compare amount/sign/absolute/net per rule.

Date | Value/posting/trade/settlement date or tolerance window.

Reference | Exact/masked/normalized reference comparison.

Currency | Same currency or approved FX/cross-currency logic.

Account | Same mapped account/control relationship.

Composite | Multiple conditions together; avoids weak one-field matching.

Tolerance | Permitted difference under governance.

Partial | Incomplete group/pending match where side missing.

Netting | Aggregate debits/credits/quantities.

Write-off | Controlled resolution for small differences.

Manual matching | User match under entitlement, limit and audit.

Stage | Purpose | Cash example ABC123

Scope | Select eligible open items | Open USD cash items for account A, value date 20-Aug, Ledger and Statement sides.

Population | Group possible candidates | Statement ABC123 and Ledger ABC123 grouped by account/currency/reference/date.

Match Rule | Compare fields | Amount 1000 = 1000, currency USD = USD, ref ABC123 = ABC123, date 20-Aug = 20-Aug.

Pass Quality | Classify quality | All mandatory exact -> perfect; date tolerance -> suggested if configured.

Proposal | Create candidate match | Proposal links statement item and ledger item with pass result.

Match | Final accepted state | Auto-perfect match or user-approved suggested match; item status matched.

Exception | Actionable unresolved break | If missing/mismatch beyond tolerance, break routed to cash investigations.

Mismatch | What happens | Senior diagnostic

Amount mismatch | Pass fails exact; may pass tolerance/write-off if allowed. | Check sign, charges, FX, partial payments, decimal conversion.

Date mismatch | May fail exact or pass date tolerance. | Check value vs posting date and timing window.

Reference mismatch | May fail population or quality. | Check normalization/narrative extraction and masking controls.

Currency mismatch | Usually fail unless cross-currency rule exists. | Check mapping/currency defaults and FX source.

Missing ledger | Statement remains unmatched/exception/pending. | Trace ledger feed/load/reject/static.

Missing statement | Ledger remains unmatched/pending until statement arrives. | Check feed timing/trigger policy.

Duplicate | Could create false many-to-one or duplicate exception. | Check duplicate file/business key.

Tolerance | Suggested/auto/write-off depending pass quality and governance. | Check limits, approval, audit.

Bank charge | Amount difference may be expected fee. | Separate fee matching/write-off/exception policy.

Timing difference | Aged outstanding until opposite side arrives. | Do not rerun/reload blindly; check trigger/feed availability.

Part 8 - Proposal Triggering

Trigger | Meaning | When to use | Risk

Any Matching Input | Proposal/match run may be considered when any configured side arrives. | Real-time/high frequency with pending visibility. | One-sided break noise if expected side is late.

All Selected/Required Inputs | Run only after all required feeds/sides are available. | Month-end statement/position/cash proof controls. | No proposal if one feed missing; need missing-feed monitoring. Always Propose | Always generate proposals when eligible data exists; exact behavior/client terms vary. | When business wants visibility even for weak/pending candidates. | Too many low-quality proposals.

Never Propose | Do not generate proposals automatically; may require manual/scheduled action. | Sensitive recons or onboarding/testing. | Items remain open if operations expect automation.

Scheduled/date-driven | Run at cut-off/date/month-end where supported/configured. | Daily ledger/monthly statement; balance proof after close. | Calendar/timezone/cutoff errors.

Feed availability | Trigger checks feed/load/balance presence. | External statement must exist before match. | Load success not enough; side may be absent.

Initiation/workflow | Item must be initiated and queued. | Controlled TLM Control workflows. | Loaded items idle if initiation fails.

Matching service | Processor must be up and consume queue. | All automated matching. | Queue backlog if service down.

Daily ledger/monthly statement scenario: Daily ledger load creates load metadata, accepted items and possibly workflow initiation. If trigger requires all selected inputs or scheduled month-end, final proposals should not be generated on daily ledger alone. The ledger items may remain open/pending or be visible as ledger-only depending design. On the 31st, statement feed arrives and loads; feed availability condition is satisfied; initiation/queue entry is created; matching service consumes queue; scope selects eligible ledger and statement items; population/groups/pass quality create proposals/matches or exceptions; balance proof checks opening + movements = closing. Load, proposal trigger, workflow initiation and matching are separate evidence points.

Part 9 - MMQ / Workflow Queue

MMQ/workflow queue must be answered as a logical processing concept unless the client confirms exact implementation. Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Topic | Senior answer

MMQ | A client/version-specific queue/message mechanism term. Conceptually it holds work for workflow/matching/integration processors.

workflow queue | Logical runtime queue of initiated work objects waiting for a processor.

initiation | Creates workflow state and/or queue entry based on load, schedule or manual event.

queue entry | Object id, type, status, priority, attempts, created timestamp, next retry.

processor/consumer | Configured workflow/matching/integration service or scheduler consumes queue. Exact service name is runbook dependent.

queue status | Ready/running/done/error/retry-like states; exact values client dependent.

retry | Failed work may be retried with count and next-run time.

poison record | A bad record that repeatedly fails and blocks or ages in queue.

stuck queue | Old ready/running/error entries, no progress, increasing backlog.

service down | Items load and queue but no proposals/matches; logs show no consumer activity.

locks | DB locks or long transactions prevent queue update/processing.

backlog/ageing | Monitor count and oldest created time by status/queue.

Who polls workflow_queue? | A configured queue consumer such as workflow processor or matching service. I verify exact process/service from the client runbook; I do not invent it.

Prove matching started | Show queue consumed/transitioned, matching run/pass results/log entries, proposals/matches/exceptions created with timestamps after load.

Part 10 - 30 TLM-Specific Verbal SQL Questions

Verbal SQL question | Placeholder SQL / spoken approach

```
File-wise matched count | SELECT b.<FILE_NAME>, COUNT(*) FROM <TLM_ITEM_TABLE> i JOIN <TLM_LOAD_BATCH_TABLE>
b ON b.<LOAD_ID>=i.<LOAD_ID> WHERE i.<MATCH_STATUS>='MATCHED' GROUP BY b.<FILE_NAME>;
File-wise unmatched count | SELECT b.<FILE_NAME>, COUNT(*) FROM <TLM_ITEM_TABLE> i JOIN
<TLM_LOAD_BATCH_TABLE> b ON b.<LOAD_ID>=i.<LOAD_ID> WHERE NVL(i.<MATCH_STATUS>,'UNMATCHED')<>'MATCHED' GROUP
BY b.<FILE_NAME>;
Match percentage | SELECT <DIM>, ROUND(100*SUM(CASE WHEN <MATCH_STATUS>='MATCHED' THEN 1 ELSE 0
END)/COUNT(*),2) FROM <TLM_ITEM_TABLE> GROUP BY <DIM>;
Break percentage | SELECT <DIM>, ROUND(100*SUM(CASE WHEN NVL(<MATCH_STATUS>,'X')<>'MATCHED' THEN 1 ELSE 0
END)/COUNT(*),2) FROM <TLM_ITEM_TABLE> GROUP BY <DIM>;
Duplicate references | SELECT <REFERENCE>, COUNT(*) FROM <TLM_ITEM_TABLE> GROUP BY <REFERENCE> HAVING
COUNT(*)>1;
Duplicate files | SELECT <FILE_NAME>, <BUSINESS_DATE>, COUNT(*) FROM <TLM_LOAD_BATCH_TABLE> GROUP BY
<FILE_NAME>, <BUSINESS_DATE> HAVING COUNT(*)>1;
Latest load | SELECT * FROM (SELECT * FROM <TLM_LOAD_BATCH_TABLE> ORDER BY <LOAD_END_TS> DESC) WHERE
ROWNUM=1;
Failed load | SELECT * FROM <TLM_LOAD_BATCH_TABLE> WHERE <STATUS> IN ('FAILED','ERROR'); -- replace status
values from client
Partial load | SELECT * FROM <TLM_LOAD_BATCH_TABLE> WHERE <EXPECTED_COUNT> <>
NVL(<ACCEPTED_COUNT>,0)+NVL(<REJECTED_COUNT>,0);
Reject count | SELECT <ERROR_CODE>, COUNT(*) FROM <TLM_REJECT_TABLE> WHERE <LOAD_ID>=:load_id GROUP BY
<ERROR_CODE>;
Items by feed | SELECT <FEED_ID>, COUNT(*) FROM <TLM_ITEM_TABLE> GROUP BY <FEED_ID>;
Items by account | SELECT <ACCOUNT_ID>, COUNT(*), SUM(<AMOUNT>) FROM <TLM_ITEM_TABLE> GROUP BY <ACCOUNT_ID>;
Items by reconciliation | SELECT <RECON_ID>, <MATCH_STATUS>, COUNT(*) FROM <TLM_ITEM_TABLE> GROUP BY
<RECON_ID>, <MATCH_STATUS>;
Proposals by status | SELECT <PROPOSAL_STATUS>, COUNT(*) FROM <TLM_PROPOSAL_TABLE> GROUP BY
<PROPOSAL_STATUS>;
Matches by day | SELECT TRUNC(<MATCH_TS>), COUNT(*) FROM <TLM_MATCH_TABLE> GROUP BY TRUNC(<MATCH_TS>);
Exceptions by type/status | SELECT <EXCEPTION_TYPE>, <STATUS>, COUNT(*) FROM <TLM_EXCEPTION_TABLE> GROUP BY
<EXCEPTION_TYPE>, <STATUS>;
Queue ageing | SELECT <QUEUE_STATUS>, COUNT(*), MIN(<CREATED_TS>) FROM <TLM_WORKFLOW_QUEUE_TABLE> GROUP BY
<QUEUE_STATUS>;
Workflow status | SELECT <WORKFLOW_STATE>, COUNT(*) FROM <TLM_WORKFLOW_TABLE> GROUP BY <WORKFLOW_STATE>;
Balance difference | SELECT <ACCOUNT>, <CCY_OR_ASSET>, SUM(<INTERNAL_BAL>)-SUM(<EXTERNAL_BAL>) diff FROM
<TLM_BALANCE_TABLE> GROUP BY <ACCOUNT>, <CCY_OR_ASSET> HAVING SUM(<INTERNAL_BAL>)<>SUM(<EXTERNAL_BAL>);
Opening + movement = closing | SELECT <ACCOUNT>, <CCY>, <OPENING_BAL>+SUM(<MOVEMENT_AMOUNT>)-<CLOSING_BAL>
proof_diff FROM <TLM_BALANCE_AND_MOVEMENT_VIEW> GROUP BY <ACCOUNT>, <CCY>, <OPENING_BAL>, <CLOSING_BAL>;
Source-to-item lineage | SELECT i.* FROM <TLM_ITEM_TABLE> i JOIN <TLM_LOAD_BATCH_TABLE> b ON
b.<LOAD_ID>=i.<LOAD_ID> WHERE b.<FILE_NAME>=:file AND i.<SOURCE_REFERENCE>=:ref;
Item-to-match lineage | SELECT i.<ITEM_ID>, m.* FROM <TLM_ITEM_TABLE> i JOIN <TLM_MATCH_TABLE> m ON
m.<MATCH_ID>=i.<MATCH_ID> WHERE i.<SOURCE_REFERENCE>=:ref;
Item-to-exception lineage | SELECT i.<ITEM_ID>, e.* FROM <TLM_ITEM_TABLE> i JOIN <TLM_EXCEPTION_TABLE> e ON
e.<ITEM_ID>=i.<ITEM_ID> WHERE i.<SOURCE_REFERENCE>=:ref;
Missing ledger vs statement | SELECT s.* FROM <TLM_ITEM_TABLE> s WHERE s.<DATA_SOURCE>='STATEMENT' AND NOT
EXISTS (SELECT 1 FROM <TLM_ITEM_TABLE> l WHERE l.<DATA_SOURCE>='LEDGER' AND l.<REFERENCE>=s.<REFERENCE> AND
l.<AMOUNT>=s.<AMOUNT>);
Wrong account mapping | SELECT <SOURCE_ACCOUNT>, <ACCOUNT_ID>, COUNT(*) FROM <TLM_ITEM_TABLE> WHERE
<LOAD_ID>=:load_id GROUP BY <SOURCE_ACCOUNT>, <ACCOUNT_ID>;
Amount mismatch candidates | SELECT l.<REF>, l.<AMOUNT> ledger_amt, s.<AMOUNT> stmt_amt FROM
<TLM_ITEM_TABLE> l JOIN <TLM_ITEM_TABLE> s ON s.<REF>=l.<REF> WHERE l.<DATA_SOURCE>='LEDGER' AND
s.<DATA_SOURCE>='STATEMENT' AND NVL(l.<AMOUNT>,0)<>NVL(s.<AMOUNT>,0);
Items loaded but not initiated | SELECT i.* FROM <TLM_ITEM_TABLE> i LEFT JOIN <TLM_WORKFLOW_TABLE> w ON
w.<ITEM_ID>=i.<ITEM_ID> WHERE w.<ITEM_ID> IS NULL AND i.<LOAD_ID>=:load_id;
Proposal without match | SELECT p.* FROM <TLM_PROPOSAL_TABLE> p LEFT JOIN <TLM_MATCH_TABLE> m ON
m.<PROPOSAL_ID>=p.<PROPOSAL_ID> WHERE m.<PROPOSAL_ID> IS NULL;
Open aged exceptions | SELECT <AGE_BUCKET>, COUNT(*) FROM <TLM_EXCEPTION_AGEING_VIEW> WHERE <STATUS> NOT IN
('CLOSED','CANCELLED') GROUP BY <AGE_BUCKET>;
Latest transaction per account | SELECT * FROM (SELECT i.*, ROW_NUMBER() OVER(PARTITION BY <ACCOUNT_ID>
ORDER BY <VALUE_DATE> DESC, <CREATED_TS> DESC) rn FROM <TLM_ITEM_TABLE> i) WHERE rn=1;
```

To discover actual physical tables: search client runbook/support SQL, inspect SmartSchema metadata, query data dictionary for known business columns, review application logs for SQL/object names if exposed, ask DBA/vendor documentation, and validate with a small known transaction. Never guess names in interview.

Part 11 - MI Deep Dive Without Inventing Syntax

MI syntax varies and is proprietary/client/version dependent. Physical object name is implementation/version/client dependent - verify against the client schema/configuration. The defensible answer is to explain conceptual structure and testing.

MI conceptual example | Source -> Target -> Transformation -> Lookup -> Validation -> Error action
 CSV cash ledger | SOURCE: pipe-delimited ledger file with account, ccy, dr_cr, amount, value_date, ref.
 TARGET: cash item fields. TRANSFORMATION: trim/ref uppercase, date parse, decimal conversion, sign normalization. LOOKUP: account alias, currency, transaction type. VALIDATION: mandatory account/currency/amount/date/ref, trailer count. ERROR ACTION: reject line with reason; partial load if policy allows.
 MT940 statement | SOURCE: SWIFT statement message. TARGET: statement-side cash item and balance fields.
 TRANSFORMATION: parse statement account/date/balances/transaction lines, normalize debit/credit, extract bank reference/narrative. LOOKUP: statement account to TLM account, currency, bank/source. VALIDATION: message structure, sequence/date, opening/closing balance control. ERROR ACTION: reject message or records depending severity and reload after correction.
 Double-entry ledger normalization | SOURCE: GL entries with debit/credit legs. TARGET: recon-eligible cash comparison item, usually only cash/nostro leg or approved derived/net item. TRANSFORMATION: select eligible GL account, normalize debit/credit sign, group/net multiple postings if configured. LOOKUP: GL account to TLM cash account, product/transaction type. VALIDATION: balanced posting if required, no duplicate source key. ERROR ACTION: reject or route accounting-control exception; do not match both GL legs blindly.

MI topic | Deep answer

Datatype conversion | String to number/date; preserve raw value for rejects/audit where possible.
 Amount conversion | Decimal scale, comma/point separator, sign from DR/CR, absolute/original amount preservation.
 Date conversion | Source format and timezone/business-date rules; invalid dates reject.
 Lookup | Account/security/currency/source aliases with effective date; lookup miss is a controlled reject or exception.
 Default | Only approved defaults for optional fields; do not default mandatory control data silently.
 Conditional logic | Different mapping based on transaction type, message type, account, debit/credit.
 Reject | Record-level reject with field/reason/line; full batch reject for severe header/trailer/parser errors.
 Partial load | Accepted records commit while rejects are logged if client policy supports it.
 Full batch failure | No records committed when critical file/header/DB/parser failure occurs.
 Testing | Valid file, invalid field, lookup miss, duplicate, partial, zero-byte, trailer mismatch, reload.
 Promotion | Config export/import or controlled deployment process; exact mechanism client dependent; require SIT/UAT and rollback.

Part 12 - DMW, MI and Informatica Comparison

Area | DMW | MI | Informatica

Primary role | Data mapping/transformation layer used in your experience | TLM message loading/integration configuration | Enterprise ETL platform

Source/target | Source files/tables to TLM target/staging/business fields | Inbound message/file to TLM

business data | Many sources to many targets

Transformation | Rules/SQL/lookup/date/amount normalization | Loader mapping/enrichment/lookups |

Expression/filter/router/joiner/aggregator

Validation/reject | Control counts, mandatory fields, rejects | MI validations and rejects |

Session/workflow errors/reject targets

Testing | Source count vs target count, field-level checks | Load/reject/item checks | Session logs, target counts, reject tables

Honest answer | Hands-on strength | Conceptual/hands-on if used in TLM | Understand concepts; do not

overclaim deep hands-on if not used

Part 13 - Informatica Honest 5-Year Answer

Spoken answer: My direct hands-on ETL exposure is SQL plus DMW/data mapping around TLM. I have not claimed deep Informatica production ownership unless the project used it. However, I understand Informatica concepts: mapping, session, workflow, lookup, expression, filter, router, joiner, aggregator, update strategy, parameterization, error handling, recovery and performance. The transferable part is source-to-target mapping, transformations, validations, reject handling and control-total reconciliation.

Informatica concept | TLM/DMW transferable explanation
Mapping | Source-to-target design.
Session | Runtime execution of a mapping.
Workflow | Orchestration of sessions/tasks/dependencies.
Lookup | Static/reference enrichment such as account alias.
Expression | Derive normalized amount/date/reference.
Filter | Pass only eligible records.
Router | Split rows by transaction type/feed outcome.
Joiner | Combine streams when source SQL join is unavailable.
Aggregator | Counts/sums/control totals.
Update Strategy | Insert/update/reject target actions; important for idempotency.
Parameterization | Environment/file/date parameters.
Error handling | Reject targets/session logs/error codes.
Recovery | Restart failed workflow safely, avoid duplicate loads.
Performance | Pushdown, indexes, partitioning, source filters, avoiding row-by-row logic.

Part 14 - Tectia / SFTP / Linux

Need | Verbal Linux command

Latest file | `ls -ltr <dir> / tail`; robust: `find <dir> -type f -printf '%T@ %p\n' / sort -n / tail -1`

Permissions | `ls -l <file>; stat <file>; id <loader_user>`

File size | `ls -lh <file>; stat -c%s <file>; du -h <file>`

Line count | `wc -l <file>`

Grep reference | `grep 'ABC123' <file>; grep -n 'ABC123' <file>`

Disk | `df -h; du -sh <dir>`

Process | `ps -ef / grep <service>; top`

Directory | `pwd; ls -la; find <dir> -maxdepth 1 -type f`

Logs | `tail -100 <log>; grep -i error <log>`

Tail live | `tail -f <log>` only during active investigation

Tectia/SFTP checks: connection/key/account, remote file created, local landing complete, checksum/size, filename convention, permissions, duplicate control, partial-transfer marker/rename convention, archive/move process.

Part 15 - ServiceMix

Logical config/check | What to inspect without inventing filenames
service/bundle | Is the integration service deployed/active?
endpoint | Source/target URL/path/queue reachable?
route | File/message route enabled and moving data?
datasource | DB connectivity/pool healthy?
logging | Errors around pickup, parse, DB insert, endpoint call.
queue | Backlog, stuck messages, poison messages.
environment properties | Business date, paths, credentials references, hostnames, feed codes.
feed stops | Check route status, source file, endpoint, logs, queue backlog.
file stuck | Check permissions, route filter, naming pattern, parser failure.
DB unavailable | Datasource errors, retry/backlog, downstream load status.
config changed | Change ticket, diff/backup, rollback, post-change validation.

Part 16 - Smart Studio / TLM View / WebConnect

Tool/concept | Interview explanation

Smart Studio | Configuration/modeling tool area used for solution setup depending TLM version/client.

Recon Admin | Administrative setup for recon/static objects depending deployment.

TLM Control | Workflow/BPM configuration and runtime lifecycle.

TLM View | Browser UI/dashboards for operations, management, audit and exceptions.

WebConnect | Web access/integration component term; exact role client dependent.

SmartSchema | Metadata layer over business/database structures; helps tailor data model.

Task | Step-by-step conceptual answer

Create dashboard | Define audience and dataset, choose recon/status fields, columns, filters, grouping/sorting, roles, test with users.

Add column | Expose existing SmartSchema/business attribute in widget/view; verify data populated and user role access.

Change header | Change dashboard column label/config if tool supports; no DB schema change unless data model needs it.

Filter | Add default filters for recon/account/status/business date/queue/age; document criteria.

Sorting | Sort by SLA/age/amount/value date/priority.

Grouping | Group by account, currency, exception type, owner, age bucket.

Role/security | Assign view/widget/inbox permissions to roles/groups; test with impacted user.

Missing records | Check filters, role, recon/status, data load, workflow, cache/index if applicable.

Backend count differs UI | Reproduce UI filter exactly in SQL; compare admin vs user view.

User cannot see view | Check role/group, dashboard sharing, inbox/workfolder access, environment URL.

Part 17 - Position Reconciliation

Position recon compares internal holdings/positions with custodian/depot positions. Key dimensions: account/depot, security/ISIN, quantity/face amount, trade date, settlement date, settled/unsettled bucket, opening position, movements, closing position, corporate actions, splits, mergers and transfers. The logical Position Balance Table/area stores position balance by account/depot/security/business date and bucket. Physical name is client dependent. A position can break even when transactions match if opening balance, settlement bucket, corporate action, transfer or security alias is wrong.

Part 18 - Exception / Break Management

MATCH -> NO MATCH -> BREAK -> EXCEPTION -> QUEUE -> OWNER -> INVESTIGATION -> COMMENT -> SUPPORTING EVIDENCE -> RESOLUTION -> APPROVAL IF REQUIRED -> CLOSE -> AUDIT. A senior answer includes ageing, SLA, owner/inbox, escalation, reason code, resolution code, notes/attachments, maker-checker controls and automatic closure when subsequent matching clears the break if configured.

Page 21

[illegible]

Part 20 - Interview Question Bank with Flagship Answers

Flagship Question: Tell me about yourself.

20-second answer: I am a TLM techno-functional consultant focused on reconciliations, SQL, data mapping and production support.

1-minute answer: I work across requirement analysis, static setup, feed/mapping validation, SQL investigation, matching support, exception analysis, SIT/UAT and production incident resolution. My strongest practical areas are SQL, DMW/data mapping and TLM reconciliation support.

Senior 5+ year answer: For a 5+ year level answer I position myself as someone who can trace a transaction end-to-end: source file, feed, load, item, recon association, workflow, queue, matching, exception and TLM View. I am careful not to overclaim tool exposure; where Informatica or client-specific MI syntax is involved, I explain transferable ETL knowledge and verify exact implementation.

Realistic cash example: In cash recon, I can take reference ABC123 from a bank statement, find it in the file, confirm load counts, item creation, matching/proposal or exception, and explain the break to operations.

Technical deep dive: This answer should sound delivery-oriented: implementation + support + testing + stakeholder communication.

DB investigation: Check known transaction through load/item/match/exception logical areas.

SQL approach:

```
SELECT * FROM <TLM_ITEM_TABLE> WHERE <SOURCE_REFERENCE>=:ref;
```

Troubleshooting: If asked for a production issue, describe layer-by-layer investigation.

Follow-up questions: Which TLM tools have you used hands-on?, What was your SQL role?, Have you used Informatica?

Interviewer trap: Overclaiming proprietary tools.

What NOT to say: Do not say you used Informatica or edited ServiceMix files if not true.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Create a new reconciliation from scratch.

20-second answer: Gather requirements, define sources/static, map feeds, configure rules/workflow/views, test and deploy.

1-minute answer: I start with business objective, sources, fields, frequency, accounts, controls, match rules, exceptions and reporting. Then I build static setup, mapping, load validation, matching, triggers, workflow and dashboards, followed by unit/SIT/UAT and production monitoring.

Senior 5+ year answer: A senior answer includes source profiling, control totals, SmartSchema/field model, set/recon/class/data source/feed/account setup, DMW/MI mapping, scope/population/pass quality/proposal trigger, workflow/exception design, TLM View, regression tests, deployment rollback and support runbook.

Realistic cash example: Cash recon ledger vs statement with account/currency/value date/ref/amount, opening+movement=closing proof.

Technical deep dive: End-to-end design with controls and evidence.

DB investigation: Verify config metadata, then load sample and trace item lifecycle.

SQL approach:

```
SELECT <RECON_ID>, <DATA_SOURCE>, COUNT(*) FROM <TLM_ITEM_TABLE> WHERE <LOAD_ID>=:load_id
GROUP BY <RECON_ID>, <DATA_SOURCE>;
```

Troubleshooting: If no match, isolate static, trigger, queue or rule failure.

Follow-up questions: Mandatory fields?, How test?, How deploy?

Interviewer trap: Jumping straight to match rules without source/static controls.

What NOT to say: Do not ignore workflow/proposal triggering.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Mandatory fields for static setup.

20-second answer: Unique codes/names, active/effective status, class, data sources, accounts, feeds, workflow and rules are logically mandatory.

1-minute answer: For set/recon: code/name, class, active dates, data sources, account eligibility, workflow, match rules, trigger. For feed: feed code, source, message type, file pattern/endpoint, mapping, target data source, duplicate/reject controls. For account: account id/alias, currency/BU/effective dates/recon link.

Senior 5+ year answer: At senior level I separate screen-mandatory from process-mandatory. Exact field labels vary by client, but a working recon needs identifiers, effective status, ownership/security, source/feed mapping, account mapping, rule/workflow/trigger configuration and audit approval.

Realistic cash example: Ledger account alias must map to cash account A; otherwise the file can load but item rejects or associates incorrectly.

Technical deep dive: Mandatory means required for controlled processing, not just UI required fields.

DB investigation: Query active/effective static by code and compare to item attributes.

SQL approach:

```
SELECT * FROM <TLM_STATIC_ACCOUNT_TABLE> WHERE <SOURCE_ACCOUNT>=:acct AND <ACTIVE_FLAG>='Y';
```

Troubleshooting: If static wrong, fix via approved config process and retest with sample load.

Follow-up questions: Which tables?, What if wrong account?

Interviewer trap: Saying a generic 'name and description'.

What NOT to say: Do not invent exact static table names.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Informatica exposure.

20-second answer: My direct hands-on ETL exposure is SQL and DMW; I understand Informatica concepts and can adapt.

1-minute answer: I know mappings, sessions, workflows, lookups, expressions, filters, routers, joiners, aggregators, update strategy, parameters, error handling and recovery. I should not claim deep Informatica production development unless I actually did it.

Senior 5+ year answer: A strong answer maps Informatica to DMW/MI: source-to-target mapping, transformations, reference lookups, validations, reject handling, control totals and restartability are common patterns.

Realistic cash example: Account alias lookup in DMW is like Informatica Lookup; amount sign derivation is like Expression; reject file is like error handling.

Technical deep dive: Honesty plus transferable ETL knowledge.

DB investigation: Use source-vs-target SQL counts to prove mapping quality.

SQL approach:

```
SELECT COUNT(*), SUM(<AMOUNT>) FROM <SOURCE_STAGE> UNION ALL SELECT COUNT(*), SUM(<AMOUNT>) FROM <TLM_ITEM_TABLE> WHERE <LOAD_ID>=:load_id;
```

Troubleshooting: If mapping fails, compare raw source, transformation and target value.

Follow-up questions: What is lookup?, What is workflow?

Interviewer trap: Pretending to be an Informatica expert.

What NOT to say: Do not fabricate direct Informatica experience.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: MI syntax.

20-second answer: Exact MI syntax is client/version dependent; I explain conceptual MI structure and verify actual syntax from client documentation/export.

1-minute answer: MI maps source files/messages to TLM business fields with transformations, lookups, validations and reject handling. The conceptual structure is source definition, record layout, field mapping, transformation, validation, target association and error action.

Senior 5+ year answer: Senior answer: never invent syntax. I can read existing MI definitions, compare with source format, test with positive/negative files, inspect rejects and trace item values. For syntax changes I follow client deployment/version control and SIT/UAT.

Realistic cash example: CSV ledger account_no -> account_id lookup; dr_cr+amount -> normalized amount; value_date parse; ref trim/uppercase; reject lookup miss.

Technical deep dive: Syntax is less important than explaining mapping lifecycle, validation and evidence.

DB investigation: Query load/reject/item values by file/ref.

SQL approach:

```
SELECT * FROM <TLM_REJECT_TABLE> WHERE <LOAD_ID>=:load_id;
```

Troubleshooting: If MI fails, check parser, datatype, lookup, mandatory fields, duplicate controls.

Follow-up questions: Can you create MI independently?, How test?

Interviewer trap: Inventing MI syntax.

What NOT to say: Do not write fake MI commands.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Can you create MI independently?

20-second answer: I can build/analyse MI mapping if I have source spec, target fields, static, validation rules and client tooling access.

1-minute answer: I need source file/message layout, target SmartSchema/business fields, account/source reference data, data source/recon setup, reject policy, duplicate controls, test files and deployment process.

Senior 5+ year answer: At senior level I say I do not create it in isolation. MI depends on static setup, target model, security, environment config, testing and approvals. I can create or modify mapping within the client tool/process and prove it with controlled test loads.

Realistic cash example: Create cash ledger MI from CSV to Ledger side with account lookup and sign normalization.

Technical deep dive: Dependencies and governance matter.

DB investigation: Validate load metadata, rejects, item values and recon association.

SQL approach:

```
SELECT <SOURCE_REF>, <ACCOUNT>, <AMOUNT>, <VALUE_DATE> FROM <TLM_ITEM_TABLE> WHERE  
<LOAD_ID>=:load_id;
```

Troubleshooting: Mapping defect: compare source raw and target canonical values.

Follow-up questions: What components required?

Interviewer trap: Saying yes without dependencies.

What NOT to say: Do not claim independent production deployment without controls.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: TLM View dashboard / change column header.

20-second answer: TLM View dashboards expose reconciliation/workflow/exception data with filters, columns and role access.

1-minute answer: To create/change a dashboard I define audience, data set, columns, filters, sorting/grouping, actions and role access. Changing a column header is a UI/dashboard metadata change if the underlying field already exists; not necessarily a DB change.

Senior 5+ year answer: Senior answer includes comparing UI counts to DB under same filters, testing as target user, checking security/inbox/workfolder access and avoiding confusion between display label and backend field name.

Realistic cash example: Cash breaks dashboard with account, currency, ref, amount, age, owner, reason, resolution.

Technical deep dive: Dashboard config sits above data/security/workflow.

DB investigation: Compare dashboard criteria with SQL count.

SQL approach:

```
SELECT COUNT(*) FROM <TLM_EXCEPTION_TABLE> WHERE <RECON_ID>=:recon AND <STATUS>='OPEN';
```

Troubleshooting: If user can't see records, check filter, role, data status and dashboard metadata.

Follow-up questions: Backend count differs?

Interviewer trap: Assuming UI missing means data missing.

What NOT to say: Do not invent TLM View metadata table names.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Recon Class vs Data Source.

20-second answer: Class is the recon model/type; data source is the input side.

1-minute answer: Cash class defines cash behavior; data sources are Ledger and Statement. Position class defines asset/quantity behavior; data sources are Internal Holding and Custodian.

Senior 5+ year answer: Senior answer: class controls semantic dimensions and proof behavior; data source controls lineage/side separation. Wrong class gives wrong behavior; wrong data source means good records are compared/routed incorrectly.

Realistic cash example: Cash class + Ledger/Statement sides.

Technical deep dive: Hierarchy and responsibility are different.

DB investigation: Count items by recon/class and data source side.

SQL approach:

```
SELECT <DATA_SOURCE>, COUNT(*) FROM <TLM_ITEM_TABLE> WHERE <RECON_ID>=:recon GROUP BY <DATA_SOURCE>;
```

Troubleshooting: If one side missing, check feed-to-data-source mapping.

Follow-up questions: Why both required?

Interviewer trap: Calling both the same.

What NOT to say: Do not call data source the physical DB.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Scope vs Population vs Pass Quality.

20-second answer: Scope selects eligible items, population groups candidates, pass quality decides whether the candidate is good enough.

1-minute answer: Scope is the universe filter; population is candidate grouping; match rule compares; pass quality categorizes perfect/suggested/pending/no-match.

Senior 5+ year answer: Senior answer: don't put all match conditions in scope, because you lose diagnostics, tolerance, hierarchy and quality categorization. Scope should control eligibility/performance; pass quality protects auto-match risk.

Realistic cash example: Scope open USD account A; population ABC123; pass quality exact amount/date/ref/currency.

Technical deep dive: This is the core of matching design.

DB investigation: Check candidate counts and pass results by run/item.

SQL approach:

```
SELECT <PASS_NAME>, <RESULT>, COUNT(*) FROM <TLM_MATCH_PASS_RESULT_TABLE> WHERE  
<RUN_ID>=:run GROUP BY <PASS_NAME>, <RESULT>;
```

Troubleshooting: If no match, determine no candidates vs failed quality.

Follow-up questions: Why not all conditions in scope?

Interviewer trap: Confusing scope with match rule.

What NOT to say: Do not invent pass result table name.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Proposal triggering / daily ledger monthly statement.

20-second answer: Load is not trigger, trigger is not matching, and matching depends on workflow/service.

1-minute answer: If ledger arrives daily and statement monthly, configure all-required or scheduled month-end trigger if final proposals should wait for statement. Daily ledger loads items; statement arrival/cutoff satisfies feed availability; workflow/queue starts matching.

Senior 5+ year answer: Senior answer separates load, initiation, queue, trigger, matching service, proposals, matches, exceptions and balance proof. Also monitor missing statement and avoid false break storm.

Realistic cash example: Ledger daily items stay open/pending until 31st statement MT940/camt arrives; then proposals run.

Technical deep dive: Feed timing and proposal policy must match business expectation.

DB investigation: Check both feed loads, initiation, queue, match results.

SQL approach:

```
SELECT <FEED_ID>, <STATUS>, COUNT(*) FROM <TLM_LOAD_BATCH_TABLE> WHERE <BUSINESS_DATE>=:bd  
GROUP BY <FEED_ID>, <STATUS>;
```

Troubleshooting: If no proposal, check required side, trigger config, queue and service.

Follow-up questions: Does load always trigger matching?

Interviewer trap: Saying matching always runs after load.

What NOT to say: Do not confuse scheduler pickup with proposal trigger.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Backend tables / tables impacted by data load.

20-second answer: Answer by logical areas, not fake table names.

1-minute answer: Static affects config/reference/audit. File load affects file metadata, load control/detail, rejects, business items and maybe workflow/initiation. Matching affects pass/proposal/match/status/exception/audit.

Senior 5+ year answer: Senior answer: first identify layer where failure occurred. Which physical table first depends on persistence/staging architecture. I prove with load id, file id, item id, workflow id, proposal/match/exception id lineage.

Realistic cash example: A cash ledger file loads: load batch success, 1000 items, 5 rejects, no matches until statement side arrives.

Technical deep dive: Layered evidence beats table memorization.

DB investigation: Trace by file/ref/load/item/recon/workflow/match/exception.

SQL approach:

```
SELECT * FROM <TLM_LOAD_BATCH_TABLE> WHERE <FILE_NAME>=:file;
```

Troubleshooting: If interviewer demands table name, state client dependent and describe how to verify.

Follow-up questions: Which table first?

Interviewer trap: Inventing physical names.

What NOT to say: Do not claim universal TLM schema.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Who polls workflow_queue?

20-second answer: A configured queue consumer such as workflow processor or matching service, exact name client dependent.

1-minute answer: The queue holds work after initiation. A processor/service/scheduler polls or consumes it, transitions workflow and can invoke matching. I verify service name from runbook/logs.

Senior 5+ year answer: Senior answer: prove with queue status/age, processor logs, matching run creation and result timestamps. If service down, loads can succeed while queue backlog grows and no proposals appear.

Realistic cash example: Ledger item queued for matching; matching service consumes queue and creates proposal.

Technical deep dive: Queue is a logical decoupling point.

DB investigation: Query queue by status/age and match results after queue consumption.

SQL approach:

```
SELECT <STATUS>, COUNT(*), MIN(<CREATED_TS>) FROM <TLM_WORKFLOW_QUEUE_TABLE> GROUP BY <STATUS>;
```

Troubleshooting: If stuck, check service, locks, poison records, retries and backlog.

Follow-up questions: What is poison record?

Interviewer trap: Naming a fake service/table.

What NOT to say: Do not invent MMQ implementation.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: File-wise matched SQL.

20-second answer: Join item/business data to load metadata, filter matched, group by file.

1-minute answer: I will replace placeholders from verified schema. Query: select file_name, count(*) from item join load on load_id where match_status='MATCHED' group by file_name.

Senior 5+ year answer: Senior answer includes knowing status values are client-specific, load-id lineage must be verified, and one file can create multiple message batches or sides depending architecture.

Realistic cash example: Count matched ledger records from CASH_LEDGER_20260831.dat.

Technical deep dive: Verbal SQL must be schema-neutral.

DB investigation: Use load/file metadata and item match status.

SQL approach:

```
SELECT b.<FILE_NAME>, COUNT(*) AS matched_count FROM <TLM_ITEM_TABLE> i JOIN  
<TLM_LOAD_BATCH_TABLE> b ON b.<LOAD_ID>=i.<LOAD_ID> WHERE i.<MATCH_STATUS>='MATCHED' GROUP  
BY b.<FILE_NAME>;
```

Troubleshooting: If count wrong, check filters, source side, duplicate loads, status values.

Follow-up questions: SQL Server equivalent?

Interviewer trap: Assuming actual column names.

What NOT to say: Use placeholders until schema verified.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Latest file Linux command / new feed directory creation.

20-second answer: Use ls/find for latest file; create directories only via approved deployment/runbook.

1-minute answer: Latest file: ls -ltr <dir> | tail or find sorted by timestamp. For new feed directory, mkdir -p path, set ownership/permissions, verify loader user access, but in production follow change control.

Senior 5+ year answer: Senior answer includes disk space, permissions, user/group, naming convention, archive/error directories and rollback. Never create production directories casually.

Realistic cash example: New cash statement feed landing/in-progress/archive/reject directories.

Technical deep dive: Linux evidence supports file layer investigation.

DB investigation: DB only after file pickup; before that use filesystem logs.

SQL approach:

```
-- Not SQL; verify later in load metadata by file name
```

Troubleshooting: If file not loaded after directory creation, check ownership, permissions, route config and naming pattern.

Follow-up questions: chmod? chown?

Interviewer trap: Running rm/mkdir in prod without approval.

What NOT to say: Do not invent exact paths.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: ServiceMix cfg files.

20-second answer: I describe logical configuration categories; exact filenames are client dependent.

1-minute answer: ServiceMix may hold service/bundle, endpoint, route, datasource, logging, queue and environment property configs. I do not name cfg files unless verified in that deployment.

Senior 5+ year answer: Senior answer: when feed stops, check route status, file path, endpoint, queue backlog, DB connectivity, logs and recent changes; restart only with runbook/approval.

Realistic cash example: Cash feed route inactive so files remain in inbound and no load batch created.

Technical deep dive: Integration layer between file/message and loader.

DB investigation: No DB load row if route never invoked loader; queue/log evidence in integration layer.

SQL approach:

```
SELECT * FROM <TLM_LOAD_BATCH_TABLE> WHERE <FILE_NAME>=:file; -- if none, check integration before DB
```

Troubleshooting: If route inactive, fix/start route under change control and validate file processed once.

Follow-up questions: Where are logs?

Interviewer trap: Inventing filenames.

What NOT to say: Do not claim universal ServiceMix config paths.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

Flagship Question: Position balance table.

20-second answer: It is a logical balance/proof area for position quantities by account/depot/security/date/bucket.

1-minute answer: Physical table name is client dependent. It stores opening/closing position, movements and buckets such as settled/unsettled where configured. It proves opening + movements = closing and compares internal vs custodian.

Senior 5+ year answer: Senior answer: transaction matching can be correct while position balance breaks due to opening balance, corporate action, transfer, security alias or settled/unsettled bucket mismatch.

Realistic cash example: Internal 10,000 ISIN X vs custodian 9,500 settled + 500 pending; total same but settled bucket breaks.

Technical deep dive: Position is quantity/security/depot oriented, not just cash amount.

DB investigation: Query balance by account/security/date/bucket and movements by item status.

SQL approach:

```
SELECT <ACCOUNT>, <SECURITY_ID>, <BUCKET>, <INTERNAL_QTY>-<EXTERNAL_QTY> diff FROM <TLM_BALANCE_TABLE> WHERE <BUSINESS_DATE>=:bd;
```

Troubleshooting: If break, check movements, CA, settlement status, aliases.

Follow-up questions: Can matched txns still break balance?

Interviewer trap: Naming physical table.

What NOT to say: Do not treat position as simple cash recon.

Confidence/accuracy note: Physical object name is implementation/version/client dependent - verify against the client schema/configuration.

50 additional senior follow-up questions

1. How will you prove the file loaded?
2. How will you prove the item was created?
3. How will you prove workflow initiated?
4. How will you prove matching started?
5. How will you distinguish load failure from matching failure?
6. What if accepted+rejected does not equal source count?
7. What if UI count differs from DB count?
8. What if one user can see data and another cannot?
9. What if load succeeded but no recon association?
10. What if two feeds map to same data source?
11. What if cash amount signs are opposite?
12. What if reference exists in narrative only?
13. What if statement arrives monthly?
14. What if queue retries keep increasing?
15. What is a poison record?
16. What if matching service is down?
17. What if ServiceMix route is inactive?
18. What if Tectia delivered a partial file?
19. What if filename has wrong business date?
20. What if header account differs from detail account?
21. What if duplicate file arrives with different name?
22. What if duplicate transaction reference is legitimate?
23. How do you tune low match rate safely?
24. How do you prevent false matches?
25. How do you test tolerance?
26. How do you test one-to-many?
27. How do you handle bank charges?
28. How do you handle write-off approvals?
29. How do you handle manual match audit?
30. How do you handle reversal?
31. How do you handle timing differences?
32. What is opening + movement = closing?
33. Why can balance break if txns match?
34. How do you investigate wrong currency?
35. How do you investigate wrong date?
36. How do you investigate wrong account?
37. How do you investigate wrong data source?
38. How do you find latest file?
39. How do you check permissions?
40. How do you check disk?
41. How do you communicate incident impact?
42. How do you avoid blind reload?
43. How do you avoid blind restart?
44. How do you validate after rerun?
45. What goes into SIT?
46. What goes into UAT?
47. What should be in support runbook?
48. What is your Informatica limitation?
49. What exact table names do you know?
50. How do you answer when exact names vary?

Final Output - Master Survival Packs

A. 100 must-remember concepts

1. Physical names are client dependent
2. Logical layer first
3. Static is not transaction data
4. Load is not matching
5. Trigger is not matching
6. Initiation creates workflow/queue
7. Queue consumer creates next state
8. Matching service must be running
9. Scope selects eligible items
10. Population groups candidates
11. Match rule compares
12. Pass quality categorizes
13. Proposal is candidate group
14. Match is final state
15. Exception is actionable break
16. Audit proves actions
17. TLM View is filtered/permissioned UI
18. SmartSchema is metadata layer
19. Recon class is model
20. Data source is side
21. Message feed is inbound stream
22. Message type is format/business category
23. File metadata proves arrival/pickup
24. Message header can drive account/date
25. Load control proves run
26. Load detail traces record
27. Rejects prove validation failure
28. BDR item is canonical record
29. Recon association makes item eligible
30. Balance proof can break even with matches
31. Nostro is our account at another bank
32. Ledger is internal record
33. Statement is external record
34. Debit/credit perspective differs
35. Sign normalization is critical
36. Mirror is not double-entry
37. Double-entry is accounting principle
38. Do not match both GL legs blindly
39. One-to-many handles aggregation
40. Tolerance needs governance
41. Manual match needs audit
42. Write-off needs approval
43. Bank charges need policy
44. FX needs agreed conversion
45. Timing difference needs ageing
46. Duplicate file needs idempotency
47. Partial load needs count controls
48. Zero byte should alert
49. Wrong filename can bypass feed
50. Wrong feed can load wrong side
51. Wrong account causes wrong route
52. Wrong data source kills matching
53. Wrong class gives wrong fields
54. Proposal trigger can be any/all/scheduled/manual
55. Daily ledger/monthly statement needs trigger design
56. Queue ageing indicates stuck processing
57. Poison record repeats failure
58. Service down causes backlog

59. DB locks can block queue
60. Logs complement SQL
61. Linux proves file state
62. Tectia is secure transfer
63. ServiceMix is integration layer
64. Do not invent ServiceMix filenames
65. MI syntax is client dependent
66. MI maps/transforms/validates
67. DMW maps source to target
68. Informatica concepts transfer
69. Be honest about Informatica
70. Unit tests prove mapping
71. SIT proves integration
72. UAT proves business acceptance
73. Regression protects match rules
74. Volume testing matters
75. Config management matters
76. Maker-checker matters
77. Role access matters
78. Dashboard labels not DB columns
79. UI missing may be filter/security
80. Backend count must mimic UI filters
81. Control totals matter
82. Source lineage matters
83. Load id matters
84. File id matters
85. Item id matters
86. Match id matters
87. Exception id matters
88. Business date matters
89. Value date differs from posting date
90. Header date may differ from filename
91. Record count excludes header/trailer
92. Accepted+rejected should reconcile
93. Rerun can duplicate
94. Archive carefully
95. Communicate impact not guesses
96. Root cause before fix
97. Validate after fix
98. Document prevention
99. SQL placeholders are acceptable
100. Data dictionary helps discover tables
101. Runbook is source of truth
102. Do not direct update prod tables
103. Ask for schema/config verification
104. Confidence comes from traceability

B. 50 backend investigation points

1. How will you prove the file loaded?
2. How will you prove the item was created?
3. How will you prove workflow initiated?
4. How will you prove matching started?
5. How will you distinguish load failure from matching failure?
6. What if accepted+rejected does not equal source count?
7. What if UI count differs from DB count?
8. What if one user can see data and another cannot?
9. What if load succeeded but no recon association?
10. What if two feeds map to same data source?
11. What if cash amount signs are opposite?
12. What if reference exists in narrative only?
13. What if statement arrives monthly?
14. What if queue retries keep increasing?

15. What is a poison record?
16. What if matching service is down?
17. What if ServiceMix route is inactive?
18. What if Tectia delivered a partial file?
19. What if filename has wrong business date?
20. What if header account differs from detail account?
21. What if duplicate file arrives with different name?
22. What if duplicate transaction reference is legitimate?
23. How do you tune low match rate safely?
24. How do you prevent false matches?
25. How do you test tolerance?
26. How do you test one-to-many?
27. How do you handle bank charges?
28. How do you handle write-off approvals?
29. How do you handle manual match audit?
30. How do you handle reversal?
31. How do you handle timing differences?
32. What is opening + movement = closing?
33. Why can balance break if txns match?
34. How do you investigate wrong currency?
35. How do you investigate wrong date?
36. How do you investigate wrong account?
37. How do you investigate wrong data source?
38. How do you find latest file?
39. How do you check permissions?
40. How do you check disk?
41. How do you communicate incident impact?
42. How do you avoid blind reload?
43. How do you avoid blind restart?
44. How do you validate after rerun?
45. What goes into SIT?
46. What goes into UAT?
47. What should be in support runbook?
48. What is your Informatica limitation?
49. What exact table names do you know?
50. How do you answer when exact names vary?

C. 30 SQL patterns

1. File-wise matched count
2. File-wise unmatched count
3. Match percentage
4. Break percentage
5. Duplicate references
6. Duplicate files
7. Latest load
8. Failed load
9. Partial load
10. Reject count
11. Items by feed
12. Items by account
13. Items by reconciliation
14. Proposals by status
15. Matches by day
16. Exceptions by type/status
17. Queue ageing
18. Workflow status
19. Balance difference
20. Opening + movement = closing
21. Source-to-item lineage
22. Item-to-match lineage
23. Item-to-exception lineage
24. Missing ledger vs statement

25. Wrong account mapping
26. Amount mismatch candidates
27. Items loaded but not initiated
28. Proposal without match
29. Open aged exceptions
30. Latest transaction per account

D. 25 Linux commands

1. ls -ltr <dir> | tail
2. find <dir> -type f -printf '%T@ %p\n' | sort -n | tail -1
3. ls -l <file>
4. stat <file>
5. wc -l <file>
6. head -20 <file>
7. tail -20 <file>
8. grep -n 'ABC123' <file>
9. grep -i error <log>
10. tail -100 <log>
11. tail -f <log>
12. df -h
13. du -sh <dir>
14. ps -ef | grep <process>
15. top
16. id <user>
17. chmod 640 <file>
18. chown user:group <file>
19. mkdir -p <dir>
20. mv <src> <dest>
21. cp <src> <dest>
22. touch <file>
23. sort <file>
24. uniq -c
25. awk -F'|' '{print \$1}' <file>

E. 25 production incidents

1. file missing
2. file late
3. zero byte
4. duplicate
5. partial load
6. mapping failure
7. lookup failure
8. wrong account
9. wrong data source
10. wrong feed
11. matching not triggered
12. queue stuck
13. service down
14. proposal not generated
15. false match
16. low match rate
17. exception spike
18. view mismatch
19. balance mismatch
20. timing difference
21. duplicate item
22. wrong sign
23. wrong currency
24. wrong date
25. wrong reference

F. 25 cash reconciliation scenarios

1. perfect match
2. amount mismatch
3. value-date mismatch
4. posting-date mismatch
5. reference mismatch
6. narrative-only reference
7. missing ledger
8. missing statement
9. duplicate ledger
10. duplicate statement
11. bank charge difference
12. FX conversion difference
13. wrong sign
14. wrong currency
15. wrong account
16. reversal
17. chargeback
18. timing difference
19. late statement
20. partial ledger file
21. zero statement file
22. manual match required
23. write-off required
24. opening balance mismatch
25. closing balance mismatch

G. 15 mirror/double-entry scenarios

1. Statement credit vs ledger debit
2. Statement debit vs ledger credit
3. Ledger has DR+CR legs
4. One statement vs multiple ledger postings
5. Partial double-entry posting
6. Unbalanced source posting
7. Wrong cash leg selected
8. Both GL legs loaded to recon
9. Mirror derived item duplicate
10. Wrong sign normalization
11. Same reference on DR and CR
12. Fee leg included in net
13. Reversal and original both present
14. Timing gap between legs
15. Manual match hides double-entry issue

H. 15 proposal-trigger scenarios

1. any input immediate
2. all required inputs
3. scheduled EOD
4. month-end statement
5. manual rerun
6. always propose
7. never propose
8. missing required side
9. late feed after cutoff
10. feed loaded but inactive
11. balance feed required
12. workflow not initiated
13. queue created but not consumed

14. service down
15. trigger changed in config

I. 15 workflow/MMQ scenarios

1. queue backlog
2. old ready entries
3. running stuck
4. retry increasing
5. poison record
6. service down
7. DB lock
8. wrong priority
9. wrong queue route
10. manual requeue
11. duplicate queue entry
12. queue consumed no result
13. initiation failed
14. workflow state invalid
15. audit missing transition

J. 20 MI/DMW scenarios

1. CSV happy path
2. MT940 parse
3. double-entry normalization
4. date parse fail
5. amount decimal fail
6. account lookup miss
7. currency invalid
8. reference null
9. default optional field
10. reject mandatory null
11. partial load
12. full batch reject
13. duplicate file
14. duplicate record
15. wrong feed mapping
16. source count mismatch
17. target amount mismatch
18. promotion issue
19. rollback mapping
20. SIT regression

K. 15 ServiceMix/Tectia scenarios

1. SFTP key expired
2. network timeout
3. partial transfer
4. checksum mismatch
5. wrong landing path
6. permission denied
7. route inactive
8. endpoint unavailable
9. DB datasource down
10. queue backlog
11. config changed
12. file stuck inbound
13. archive move failed
14. duplicate delivery
15. log rotation hides error

L. 20 Smart Studio/TLM View scenarios

1. create cash dashboard
2. add exception column
3. change column label
4. filter by recon
5. filter by account
6. sort by ageing
7. group by owner
8. role access missing
9. view shared incorrectly
10. backend count differs
11. user filter saved
12. wrong business date
13. status excluded
14. workfolder missing
15. inbox access missing
16. audit column missing
17. match id column
18. reason/resolution columns
19. dashboard performance
20. export/report issue

M. 100 interview questions

1. How will you prove the file loaded?
2. How will you prove the item was created?
3. How will you prove workflow initiated?
4. How will you prove matching started?
5. How will you distinguish load failure from matching failure?
6. What if accepted+rejected does not equal source count?
7. What if UI count differs from DB count?
8. What if one user can see data and another cannot?
9. What if load succeeded but no recon association?
10. What if two feeds map to same data source?
11. What if cash amount signs are opposite?
12. What if reference exists in narrative only?
13. What if statement arrives monthly?
14. What if queue retries keep increasing?
15. What is a poison record?
16. What if matching service is down?
17. What if ServiceMix route is inactive?
18. What if Tectia delivered a partial file?
19. What if filename has wrong business date?
20. What if header account differs from detail account?
21. What if duplicate file arrives with different name?
22. What if duplicate transaction reference is legitimate?
23. How do you tune low match rate safely?
24. How do you prevent false matches?
25. How do you test tolerance?
26. How do you test one-to-many?
27. How do you handle bank charges?
28. How do you handle write-off approvals?
29. How do you handle manual match audit?
30. How do you handle reversal?
31. How do you handle timing differences?
32. What is opening + movement = closing?
33. Why can balance break if txns match?
34. How do you investigate wrong currency?
35. How do you investigate wrong date?
36. How do you investigate wrong account?
37. How do you investigate wrong data source?

38. How do you find latest file?
39. How do you check permissions?
40. How do you check disk?
41. How do you communicate incident impact?
42. How do you avoid blind reload?
43. How do you avoid blind restart?
44. How do you validate after rerun?
45. What goes into SIT?
46. What goes into UAT?
47. What should be in support runbook?
48. What is your Informatica limitation?
49. What exact table names do you know?
50. How do you answer when exact names vary?
51. File-wise matched count
52. File-wise unmatched count
53. Match percentage
54. Break percentage
55. Duplicate references
56. Duplicate files
57. Latest load
58. Failed load
59. Partial load
60. Reject count
61. Items by feed
62. Items by account
63. Items by reconciliation
64. Proposals by status
65. Matches by day
66. Exceptions by type/status
67. Queue ageing
68. Workflow status
69. Balance difference
70. Opening + movement = closing
71. Source-to-item lineage
72. Item-to-match lineage
73. Item-to-exception lineage
74. Missing ledger vs statement
75. Wrong account mapping
76. Amount mismatch candidates
77. Items loaded but not initiated
78. Proposal without match
79. Open aged exceptions
80. Latest transaction per account
81. Explain cash recon
82. Explain position recon
83. Explain mirror vs double entry
84. Explain MI testing
85. Explain DMW vs Informatica
86. Explain ServiceMix
87. Explain Tectia
88. Explain TLM View
89. Explain SmartSchema
90. Explain exception lifecycle
91. Explain balance proof
92. Explain wrong sign issue
93. Explain duplicate load
94. Explain low match rate
95. Explain production communication
96. Explain SIT
97. Explain UAT
98. Explain rollback
99. Explain config management
100. Explain stakeholder management

N. Final 1-page survival sheet

If the interviewer says which table? answer: Physical object name is implementation/version/client dependent - verify against the client schema/configuration. Then identify the layer: static, feed/load, reject, item/BDR, recon status, workflow/queue, matching/proposal/match, exception, balance, audit, security or dashboard metadata.

If the interviewer says what happens next? answer with sequence: load -> item -> static/recon association -> workflow initiation -> queue -> scope -> population -> pass quality -> proposal -> match/exception -> audit -> TLM View.

If the interviewer says how do you prove it? use evidence: file check, load counts, reject details, item query, recon/source/account status, workflow/queue status, matching/proposal/match result, exception status, audit and UI filter/security comparison.

If the interviewer tests bluffing, do not invent table names, MI syntax, MMQ implementation, ServiceMix filenames or direct production update steps. Say you verify exact names from client schema/runbook and explain the logical investigation.

For cash reconciliation, always mention account, currency, value date, amount, debit/credit sign, reference, statement/ledger, opening balance, movements, closing balance, proof, timing differences, bank charges, tolerance and audit.

O. 7-day learning plan

1. Day 1: Master logical DB architecture and 'which table' answers with placeholders.
2. Day 2: Practice complete data flow and reverse troubleshooting from TLM View to source file.
3. Day 3: Cash reconciliation deep dive: nostro, ledger/statement, sign, balances, proof, charges, FX.
4. Day 4: Matching deep dive: scope, population, rules, pass quality, proposals and mismatch examples.
5. Day 5: Workflow/MMQ/proposal triggering: daily ledger/monthly statement, queue stuck, service down.
6. Day 6: SQL/Linux/MI/DMW drill: 30 verbal SQLs, 25 Linux commands, mapping/reject scenarios.
7. Day 7: Mock interview: answer flagship questions aloud, avoid hallucinated names, communicate incidents clearly.